

Unit 1

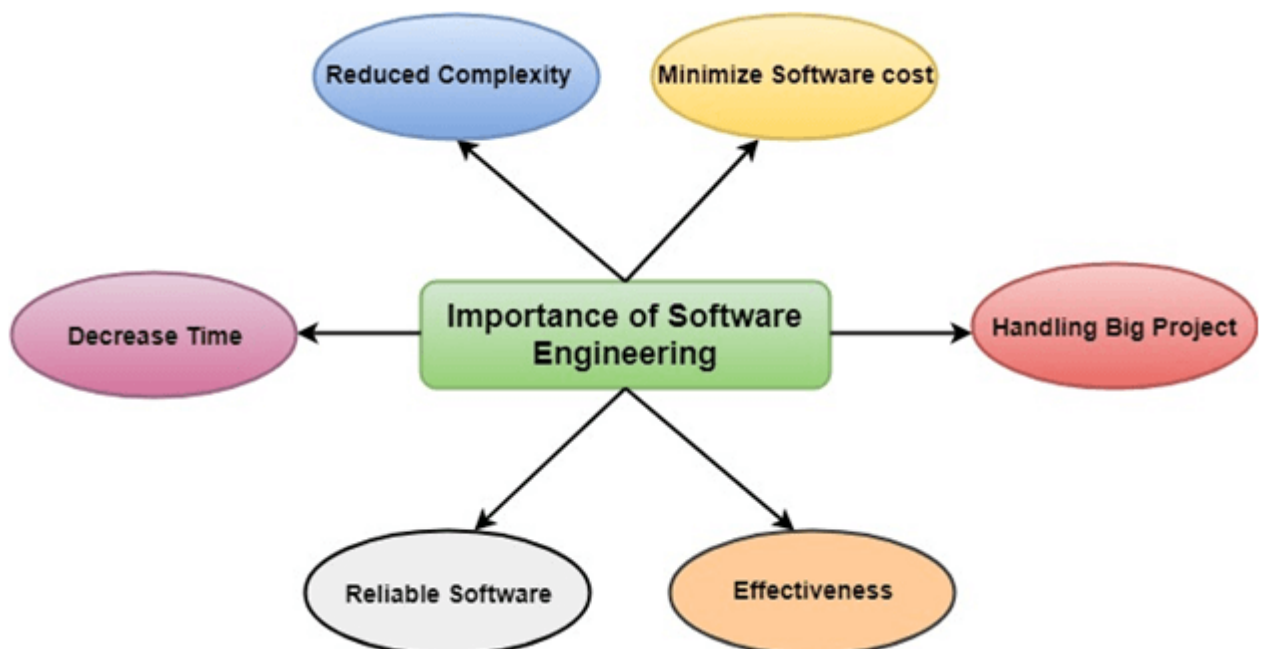
The term **software engineering** is the product of two words, software and engineering.

Software is the collection of computer programs, procedure rules and associated documentation and data.

Engineering is application of scientific and practical knowledge to invent, design, build, maintain, and improve frameworks, processes, etc.

Software Engineering is the application of a systematic, disciplined and quantifiable approach to the development, operation and maintenance of software.

Importance of Software Engineering



1. **Reduces complexity:** Big software is always complicated and challenging to progress.. Software

engineering divides big problems into various small issues. And then start solving each small issue one by one.

2. **To minimize software cost:** A lot of manpower is required to develop software with a large number of codes. But in software engineering, programmers project everything and decrease things that are not needed.
3. **To decrease time:** There is a very time-consuming procedure, and if it is not well handled, then this can take a lot of time. So if you use software engineering, then it will decrease lot of time.
4. **Handling big projects:** Big projects need lots of patience, planning, direction, testing, and maintenance and it should be completed. So to handle a big project without any problem, company go for software engineering method.
5. **Reliable software:** Software should be secure and if any bugs come in the software, the company is responsible for solving all these bugs. Because in software engineering, testing and maintenance are given, so there is no worry of its reliability.
6. **Effectiveness:** Effectiveness comes if anything has made according to the standards. Software standards are big target of companies to make it more effective. So Software becomes more effective with the help of software engineering.

Software Crisis:

- Software Crisis can be defined as a set of problems raised during the software development periods.
- Software Crisis is a term used in computer science for the difficulty of writing useful and efficient computer programs in the required time.
- If we use the same methods, and same tools after the fast increase in software demand, software complexity, and software challenges, then there arise some issues like software budget problems, software efficiency problems, software quality problems, software management and delivery problems. This condition is called a **Software Crisis**.

Factors or Reasons responsible for Software Crisis:

1. **Problem of Scale:** The ways to develop a small software having hundreds of lines will not work for large systems having thousands of lines will not work for large systems having thousands of lines.
2. **Software are expensive:** The cost to develop a software is high as compared to hardware. Highly qualified and skilled manpower is required to develop software. They have to be paid a large amount of money which increases expenses.
3. **Software are late:** It means software cannot be completed within the specified period of time. The software is not delivered in time.

4. **Software are unreliable:** It means software do not perform the required function. Many failures occur due to bugs that get introduced into the software.
5. **Changing Requirements:** Many software projects faced changing requirements. The inability to handle these changes led to project delays, cost overruns, software not aligning with user needs.
6. **Absence of Standards:** The lack of industry-wide standards and best practices resulted in inconsistent approaches to development, contributed to crisis.
7. **Poor Quality Assurance:** Inadequate testing and quality assurance processes resulted in software with numerous bugs, security vulnerabilities, and reliability issues. This undermined user confidence and overall system stability.
8. **Duplication of efforts:** Due to absence of automation in most of the software development activities the efforts have to be repeated.
9. **Lack of Communication:** Limited communication among development teams, stakeholders, and end-users led to misunderstandings and misalignments. Communication gaps cause delays.
10. **Increase in complexity of the problem area:** Demand for larger and more complex software systems increased.
11. **Inadequate Project Management:** Weak project management practices, including poor planning, resource allocation, and progress tracking, contributed to missed deadlines, budget overruns, and unsuccessful projects.

Solutions to Software Crisis

- 1. Structured Methodologies:** The use of structured methodologies, like Waterfall and Agile, brought systematic approaches to software development.
- 2. Formal Software Development:** Use of formal methods, mathematical modeling enhance software reliability by verification and validation of software
- 3. Requirements Engineering:** Emphasis on thorough requirements gathering and analysis helped manage changing requirements effectively, ensuring software alignment with user needs.
- 4. Quality Assurance and Testing:** Robust testing and quality assurance practices, such as unit testing, integration testing, and automated testing, minimized occurrence of bugs and vulnerabilities.
- 5. Code Reusability:** The development of modular programming and component-based development allowed for greater code reusability across projects, boosting productivity and reducing redundancy.
- 6. Improved Project Management:** The adoption of project management methodologies, such as Scrum lead to better resource allocation, communication, and tracking, leading to more successful projects.
- 7. Standardization and Best Practices:** The establishment of industry-wide standards, coding conventions, best practices enhance overall quality of software products.

Software Processes & Characteristics

Software Process refers to the set of activities whose goal is development or evolution of software.

There are four key activities of software processes.

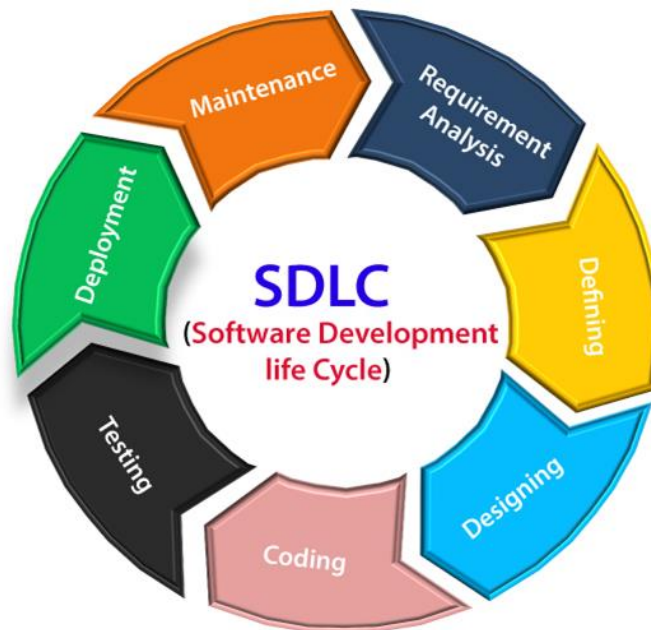
1. **Software Specifications** - In this stage, a brief description of the functionalities of the Software and its constraints must be provided.
2. **Software Development** - This is the stage of designing, programming, testing, and fixing bug.
3. **Software Validation** - This is the stage of software evaluation. It is done to ensure that the Software performs all required functions.
4. **Software Evolution** - It is the process of updating the Software to meet client changing needs.

Characteristics of Software Process

1. **Predictability**- It determines how accurately the outcome of a process in a project can be predicted.
2. **Support Testability and Maintainability**- If we want to reduce overall cost of software, the goal of development should be to reduce maintenance effort.
3. **Support Change**- A process that can handle change easily is desirable.
4. **Early Defect Removal**- Error detection and correction should be continuous process and should verify output of each phase before starting next.
5. **Process improvement and Feedback**- Process must be improved based on previous experience to increase quality and decrease cost and must feed information back to facilitate this improvement.

Software Development Life Cycle (SDLC)

A life cycle model represents all methods required to make a software product transit through its life cycle stages. SDLC represents process of developing software.



Stage1: Planning and requirement analysis

- Requirement Analysis is the most important and necessary stage in SDLC.
- Planning for the quality assurance requirements and identifications of risks associated with projects is also done at this stage.
- Once requirement is understood, the SRS (Software Requirement Specification) document is created.

Stage2: Defining Requirements

- Once the requirement analysis is done, the next stage is to document the software requirements and get them accepted from the project stakeholders.

- This is accomplished through "SRS" document which contains all product requirements to be constructed and developed during project life cycle.

Stage3: Designing the Software

- The next phase is to bring down all the knowledge of requirements, analysis, and design of software project. This phase is product of last two, like inputs from customer and requirement gathering.

Stage4: Developing the project

- In this phase of SDLC, actual development begins, and programming is built. The implementation of design begins concerning writing code. Programming tools like compilers, interpreters, debuggers are used to develop and implement code.

Stage5: Testing

- After code is generated, it is tested against the requirements to make sure that products are solving needs gathered during the requirements stage.
- During this stage, unit testing, integration testing, system testing, acceptance testing are done.

Stage6: Deployment

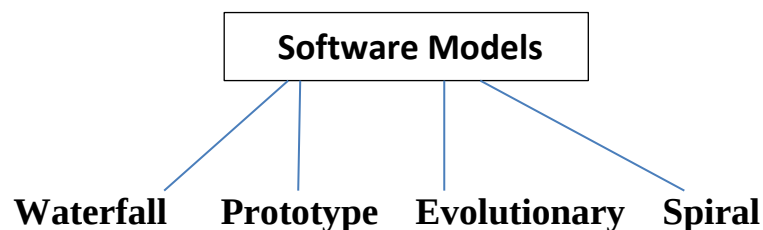
- Once the software is certified, and no bugs or errors are stated, then it is deployed. Then based on assessment, the software may be released. After the software is deployed, then its maintenance begins.

Stage7: Maintenance

- When client starts using developed systems, then the real issues come and requirements to be solved from time to time.

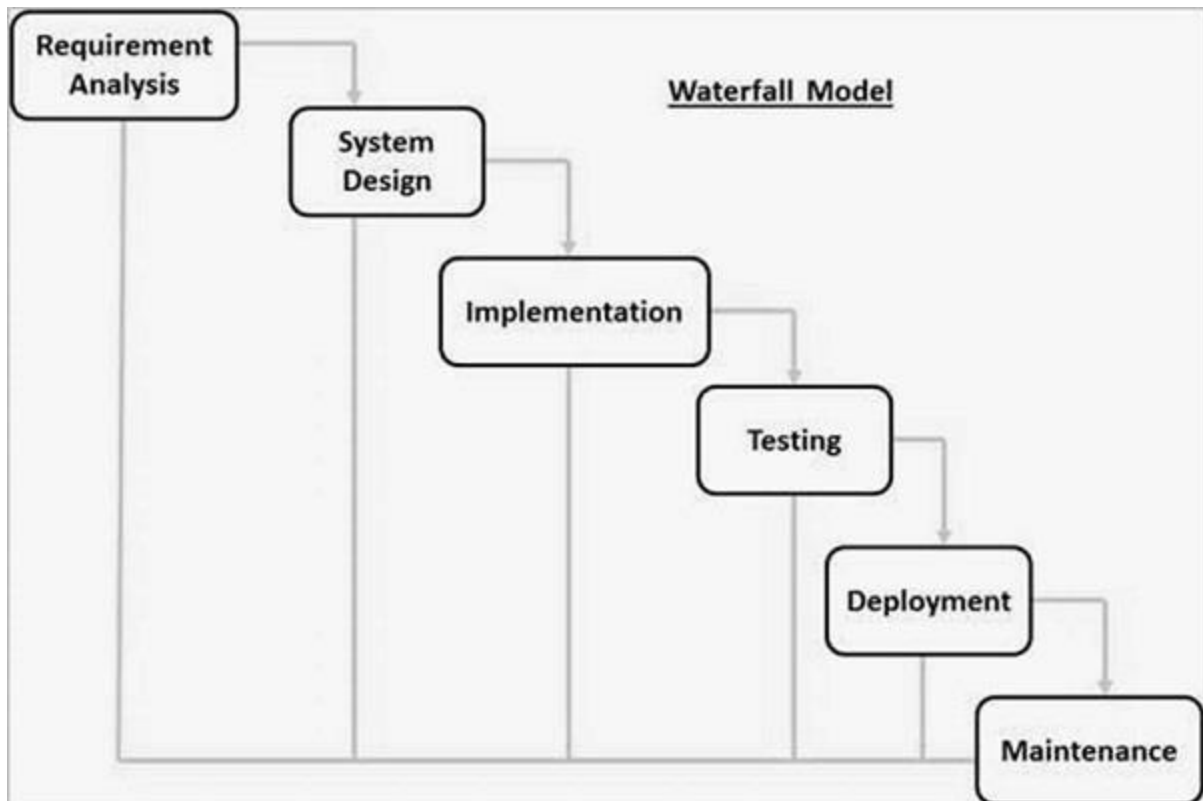
Software Life Cycle Models

- A software life cycle model (also termed process model) is a pictorial and diagrammatic representation of the software life cycle.
- Life cycle model maps various activities performed on software product from inception to retirement.
- Without software life cycle models, it becomes tough for software project managers to monitor the progress of the project.



1. Waterfall Model

- The Waterfall Model was the first Process Model.
- It is very simple to understand and use. In a waterfall model, each phase must be completed before the next phase can begin.
- It is also referred to as a **linear-sequential life cycle model** because any phase in the development process begins only if previous phase is complete.
- In waterfall model, the phases do not overlap.
- The outcome of one phase acts as the input for the next phase sequentially.
- Phases of waterfall model are performed one after other and fall like water, so called Waterfall Model.
- Mostly used for small projects.



Phases of waterfall model

- 1. Requirement Gathering and analysis** – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.
- 2. System Design** – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.
- 3. Implementation** – With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.

4. Integration and Testing – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.

System testing consists of three kinds of testing-

α testing- This system testing performed by development team.

β testing-It is the system testing performed by a friendly set of customers.

Acceptance testing: It is the system testing performed by the customer himself after the product delivery to determine whether to accept or reject delivered product.

5. Deployment of system – Once the functional and non-functional testing is done; product is deployed in the customer environment or released into the market.

6. Maintenance-software maintenance is modification of a software product after delivery to correct faults, to improve performance. It makes sure that application is running in respective environment. In case any issue is detected, it is fixed.

Advantages:

- 1. Easy to Understand:** Classical Waterfall Model is very simple and easy to understand.
- 2. Individual Processing:** Phases in the Classical Waterfall model are processed one at a time.
- 3. Properly Defined:** In the classical waterfall model, each stage in the model is clearly defined.
- 4. Clear Milestones:** Classical Waterfall model has very clear and well-understood milestones.

5. Properly Documented: Processes, actions, and results are very well documented.

Disadvantages:

1. **No Feedback Path:** In this model evolution of software from one phase to another phase is like a waterfall. It assumes that no error is committed by developers during any phase. Therefore, it does not incorporate any mechanism for error correction.
2. **Difficult to accommodate Change Requests:** The customer's requirements keep on changing with time. It is difficult to accommodate change requests after requirements specification phase is complete.
3. **No Overlapping of Phases:** This model recommends that a new phase can start only after completion of previous phase. But in to increase efficiency and reduce cost, phases may overlap.
4. **Limited Flexibility:** It is not well-suited for projects with changing requirements. Once a phase has been completed, it is difficult to make changes or go back to a previous phase.
5. **Limited Stakeholder Involvement:** Stakeholders are involved in early phases of the project (requirements gathering, analysis) but may not involve in later phases (implementation, testing, and deployment).
6. **Late Defect Detection:** Testing is typically done toward end of development process. This means that defects may not be discovered until late in the

development process, which can be expensive and time-consuming to fix.

7. Lengthy Development Cycle: This Model can result in a lengthy development cycle, as each phase must be completed before moving on to the next. This can result in delays and increased costs.

8. Not Suitable for Complex Projects: The Waterfall Model is not well-suited for complex projects, as linear and sequential nature of model can make it difficult to manage multiple dependencies and interrelated components.

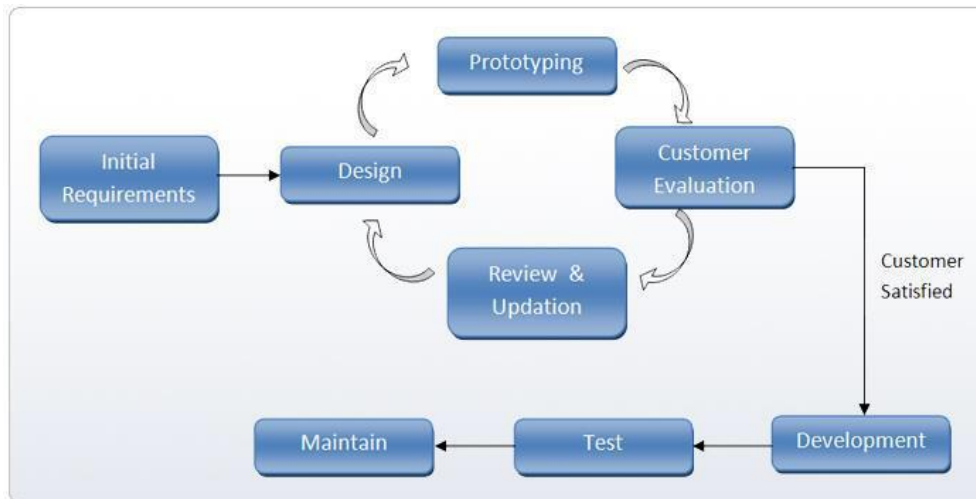
When to Use Classical Waterfall Model

- Only well-defined, unambiguous, and fixed requirements are employed with this paradigm.
- The definition of a product is constant.
- When it's a brief project.

2. Prototyping Model

- The Prototyping Model is a systems development method in which a prototype is built, tested, and then reworked as necessary until an acceptable prototype is finally achieved from which the complete system or product can now be developed.
- This model works best in where not all of project requirements are known in detail ahead of time.
- It is an iterative, trial-and-error process that takes place between the developers and the users.

- Risks associated with the projects are being reduced through the use of prototyping.
- The development of the prototype typically starts when the preliminary version of the requirements specification document has been developed.
- After prototype has been developed, end users and clients are given an opportunity to use prototype. They provide feedback to developers regarding the prototype: what is correct, what needs to be modified, what is missing, what is not needed, etc.
- Based on feedback, the prototype is modified to incorporate some of the suggested changes that can be done easily, and then the users and the clients are again allowed to use the system.



Steps Prototyping Model

Step 1: Requirement Gathering and Analysis: This is initial step in designing a prototype model. In this phase, users are asked what they expect from system.

Step 2: Quick Design: This step covers basic design of requirement through which a quick overview can be easily described.

Step 3: Build a Prototype: This step helps in building an actual prototype from the knowledge gained from prototype design.

Step 4: Initial User Evaluation: This step describes preliminary testing where investigation of performance model occurs, as the customer will tell the strength and weaknesses of design, which was sent to the developer.

Step 5: Refining Prototype: If any feedback is given by the user, then improving the client's response to feedback and suggestions, the final system is approved.

Step 6: Implement Product and Maintain: This is the final step in phase of the Prototyping Model where the final system is tested and distributed to production, here program is run regularly to prevent failures.

Advantages of Prototyping Model

- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements can be easily accommodated as there is scope for refinement.
- Missing functionalities can be easily figured out.
- Errors can be detected much earlier thereby saving a lot of effort and cost.
- The developed prototype can be reused by the developer for more complicated projects in future.
- Flexibility in design.

- Early feedback from customers can help guide the development process and ensure that the final product meets their expectations.
- Prototyping can help reduce risk of project failure by identifying issues and addressing them early.

Disadvantages of Prototyping Model

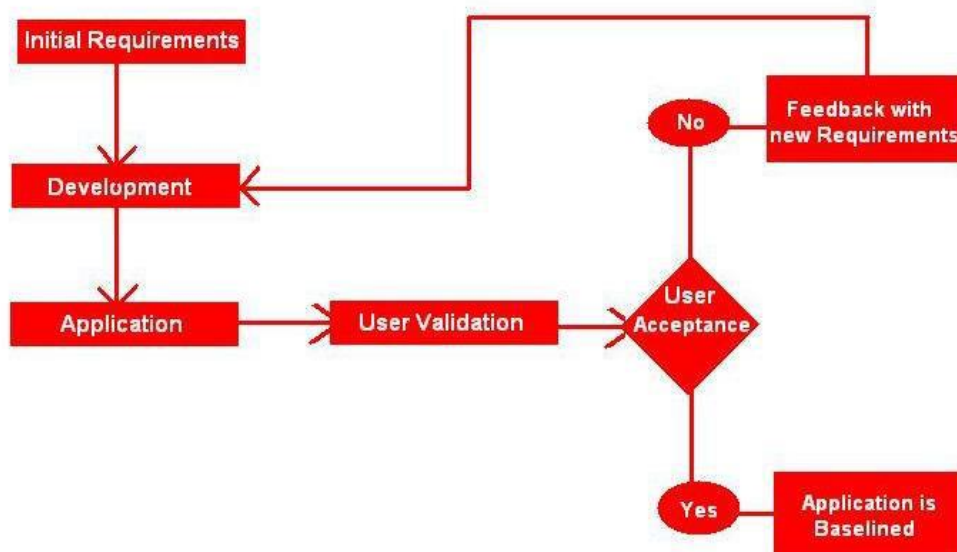
- Costly with respect to time as well as money.
- Poor Documentation due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- Uncertainty in determining number of iteration that would be required before the prototype is finally accepted by the customer.
- After seeing early prototype, customers sometimes demand actual product to be delivered soon.
- The customer might lose interest in the product if he/she is not satisfied with the initial prototype.
- The prototype may be developed using different tools and technologies, leading to additional training and maintenance costs.
- The prototype may not reflect actual business requirements of customer, leading to dissatisfaction with the final product.

Applications of Prototyping Model

- The Prototyping Model should be used when the requirements of product are not clearly understood or requirements are changing quickly.

3.Evolutionary model

- It is also called **successive versions model** or **incremental model**.
- At first, a simple working model is built.
- Subsequently it undergoes functional improvements & we keep on adding new functions till the desired system is built.
- It divides the development cycle into smaller, incremental models in which users are able to get access to product at end of each cycle.
- Feedback is provided by users on product for the planning stage of next cycle and the development team responds, often by changing the product. Therefore, software product evolves with time.
- Evolutionary model suggests breaking down of work into smaller chunks, prioritizing them and delivering those chunks to customer one by one.



Advantages:

- In evolutionary model, a user gets a chance to experiment partially developed system.
- The customer's confidence increases as he constantly gets quantifiable goods or services from beginning of project to verify his requirements.
- The model allows for changing requirements as well as all work is broken down into maintainable work chunks.
- It reduces the error because the core modules get tested thoroughly.

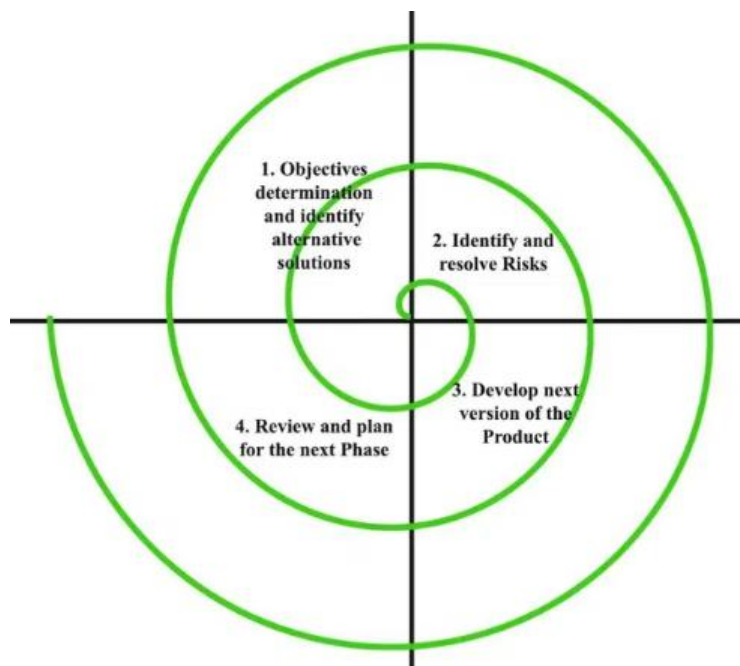
Disadvantages:

- Sometimes it is hard to divide the problem into several versions that would be acceptable to the customer which can be incrementally implemented and delivered.
- It can lead to increase complexity if there are large number of chunks.

4.Spiral Model

- **The Spiral Model** is most important Software Development Life Cycle models, which provides support for **Risk Handling**.
- It is based on idea of a spiral, with each iteration of spiral representing complete software development cycle from requirement gathering and analysis to design, implementation, testing, and maintenance.

- In its diagrammatic representation, looks like a spiral with many loops. The exact number of loops of the spiral is unknown and can vary from project to project.
- Each loop of the spiral is called a **Phase** of software development process. The exact number of phases needed to develop product can be varied by project manager.
- The Spiral model is called a **Meta-Model** because it subsumes all the other SDLC models.
- The Radius of the spiral at any point represents the cost of the project.



Each phase of the Spiral Model is divided into four quadrants. The functions of these four quadrants are:

- 1. Objectives determination and identify alternative solutions:** Requirements are gathered from the customers and the objectives are

identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.

2. **Identify and resolve Risks:** During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the Prototype is built for the best possible solution.
3. **Develop the next version of the Product:** During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.
4. **Review and plan for the next Phase:** In the fourth quadrant, the Customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.

Advantages of the Spiral Model

1. **Risk Handling:** The projects with many unknown risks that occur as the development proceeds, in that case, Spiral Model is the best development model to follow due to the risk analysis and risk handling at every phase.
2. **Good for large projects:** It is recommended to use the Spiral Model in large and complex projects.

3. **Flexibility in Requirements:** Change requests in the Requirements at a later phase can be incorporated accurately by using this model.
4. **Customer Satisfaction:** Customers can see the development of product at early phase of software development and thus, they habituated with system by using it before completion of total product.
5. **Iterative and Incremental Approach:** The Spiral Model provides iterative and incremental approach to software development, allowing for flexibility in response to changing requirements.
6. **Emphasis on Risk Management:** The Spiral Model places a strong emphasis on risk management, which helps to minimize the impact of uncertainty and risk on the software development process.
7. **Improved Communication:** The Spiral Model provides for regular evaluations and reviews, which can improve communication between the customer and the development team.
8. **Improved Quality:** The Spiral Model allows for multiple iterations of the software development process, which can result in improved software quality and reliability.

Disadvantages of the Spiral Model

1. **Complex:** The Spiral Model is much more complex than other SDLC models.
2. **Expensive:** Spiral Model is not suitable for small projects as it is expensive.

3. **Too much dependability on Risk Analysis:** The successful completion of the project is very much dependent on Risk Analysis. Without very highly experienced experts, it is going to be a failure to develop a project using this model.
4. **Difficulty in time management:** As the number of phases is unknown at the start of the project, time estimation is very difficult.
5. **Complexity:** The Spiral Model can be complex, as it involves multiple iterations of the software development process.
6. **Time-Consuming:** The Spiral Model can be time-consuming, as it requires multiple evaluations and reviews.
7. **Resource Intensive:** The Spiral Model can be resource-intensive, as it requires a significant investment in planning, risk analysis, and evaluations.

When to Use the Spiral Model

1. It is used for complex and large software development projects, as it allows for a more flexible and adaptable approach to software development.
2. It is also well-suited to projects with significant uncertainty or high levels of risk.
3. When it is appropriate to create a prototype
4. When evaluating risks and costs is crucial.

Requirement Engineering –

- The process to gather the software requirements from client, analyze, and document them is known as **requirement engineering**.
- The goal of requirement engineering is to develop and maintain ‘System Requirements Specification’ document.

Requirement Engineering Process

It is a four step process, which includes –

- i. Feasibility Study
- ii. Requirement Gathering
- iii. Software Requirement Specification (SRS)
- iv. Software Requirement Validation

1.Feasibility study

- When client approaches organization for getting desired product developed, it comes with a rough idea about what functions software must perform.
- The analysts do detailed study whether desired system and its functionality are feasible to develop.
- This study analyzes whether the software product can be practically possible in terms of implementation, cost constraints and objectives of the organization.
- The output of this phase should be a feasibility study report that should contain adequate comments and recommendations for management.

2.Requirement Gathering

- If the feasibility report is positive , next phase starts with gathering requirements from the user.
- Analysts and engineers communicate with client and end-users to know their ideas on what features they want the software to include.

3.Software Requirement Specification (SRS)

- SRS is a document created by system analyst after requirements are collected from stakeholders.
- SRS defines how software will interact with hardware, external interfaces, speed of operation, response time of system, portability of software across various platforms, maintainability, Security, Quality, Limitations etc.
- It is the responsibility of system analyst to document the requirements in technical language so that they can be comprehended and used by the software development team.

4.Software Requirement Validation

- After requirement specifications are done, the requirements mentioned in SRS are validated.
- Requirements are checked for following conditions
- If they can be practically implemented
- If they are valid and as per functionality and domain of software
- If there are any ambiguities
- If they are complete

Requirement Elicitation Technique

- Requirements elicitation is the process of gathering and defining requirements for a software system.
- The goal of requirements elicitation is to ensure that the software development process is based on a clear understanding of the customer's needs and requirements.
- Requirements elicitation involves the identification, collection, analysis, and refinement of the requirements for a software system.

1. Quality Function Deployment (QFD)

- QFD is a quality management technique that translates the need of the customer into technical requirement for software.
- QFD defines requirement in a way that maximizes the customer specification.
- It is one of the requirement gathering techniques.
- It is used to find out the requirement that enhance the quality of the software product.
- Quality in the software product is achieved by fulfilling customer's needs.
- It emphasizes an understanding of what is valuable to the customer and then deploys these values throughout the engineering process.
- It uses customer interviews and observation surveys and examination of historical data as raw data for the requirement gathering activity.

- Those data are then translated into a table of requirements and this table is called **customer voice table**.
- Customer voice table contains summary of requirements.

The QFD method has the following steps:

1. Identify all the stakeholders i.e. customers, users and developers. Also identify any constraints identified by the customers that affect requirements development.
2. List out requirements from customers.
3. A value indicating a degree of importance is assigned to each requirement. Customer determines the importance of each requirement on a scale of 1 to 5.

5 points: very important

4 points: important

3 points: average

2 points: less important

1 point: not important

QFD identifies three types of requirements:

a. Normal Requirements:

- Normal Requirements are requirements that must be present in product for customer to be satisfied.
- In other words, if these requirements are presents then the customer is satisfied.

b. Expected Requirements:

- These requirements are implicit to the product or system.

- These may be so fundamental that the customer does not explicitly state them but their absence may cause significant dissatisfaction.
- It can be easy to use or reliability of operation.
- Customer assumes these to be present in a professionally developed product without having to request them explicitly.

c. Exciting Requirements:

- These features go beyond customer's expectations.
- These prove to be very satisfying when present.

2. FAST (Facilitated Application Specification Technique)

- In this technique, a joint team of customers and developers work together to understand purpose, expectations and elements of solution.
- The team includes one expert from each field of developers and designers and experts from each department of customer's side.

Basic for FAST approach are:

- Arrange a meeting at neutral site for developers and customers.
- Establishment of rules for preparation and participation.

- Prepare an informal agenda that encourages free flow of ideas.
- Appoint a facilitator to control the meeting.
- Preparation of agenda is to cover all important points.

Activities of FAST session

- Each attendee is asked to make a list of objects, services, constraints for discussion.
- Lists are displayed in the meeting by using board, large sheet of paper etc.so that they are visible to all the participants.
- The combined list for each topic is prepared by eliminating redundant entries and adding new ideas.
- The combined lists are again discussed and consensus lists are finalized by the facilitator.
- Once consensus list is completed, team is divided into small sub teams, each work to develop mini-specifications for one or more entries of the list.
- Each sub team then presents mini-specification to all FAST attendees. After discussion, additions or deletions are made to the list.
- Each attendee prepares a list of validation criteria for the product/system and present list to the team. A consensus list of validation criteria is created.
- A sub team may be asked to write the complete draft specifications using all inputs from the FAST meeting.

Advantages of Requirements Elicitation:

- Helps to clarify and refine customer requirements.
- Improves communication and collaboration between stakeholders.
- Increases the chances of developing a software system that meets customer needs.
- Avoids misunderstandings and helps to manage expectations.
- Supports the identification of potential risks and problems early in the development cycle.
- Facilitates the development of a comprehensive and accurate project plan.
- Increases user and stakeholder confidence in the software development process.

Disadvantages of Requirements Elicitation:

- Can be time-consuming and expensive.
- Requires specialized skills and expertise.
- May be impacted by changing business needs and requirements.
- Can be impacted by political and organizational factors.
- Can result in a lack of buy-in and commitment from stakeholders.
- May result in incomplete or inaccurate requirements if not properly managed.
- Can lead to increased development costs and decreased efficiency if requirements are not well-defined.

Requirements Analysis

Requirement analysis is significant and essential activity after elicitation. This activity reviews all requirements and may provide a graphical view of the entire system. Here, we may also use the interaction with customer to clarify points of confusion and to understand which requirements are more important.

Data Flow Diagram (DFD)

- Data flow diagram is graphical representation of flow of data in an information system. It is capable of depicting incoming data flow, outgoing data flow and stored data.
- DFD show all the essential components and have they fit together in the system.
- The main advantage of DFD is that they are easily understood by the users, and hence users can suggest modifications in the proposed system.

The main Goals of DFD are:-

- To provide an overview of that data a system would process.
- To provide hierarchical break down of the system.
- To show what files are used and where the result flows.
- To document information flow within the system.

Symbols used for constructing DFD's:-



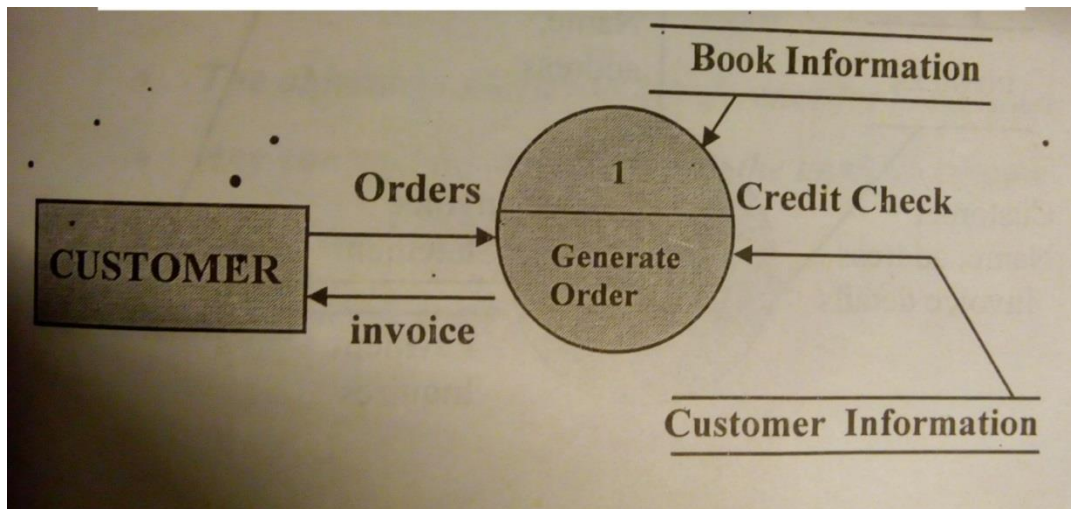
- **Entities** - Entities are source and destination of information data. Entities are represented by rectangles with their respective names.
- **Process** - Activities and action taken on the data are represented by Circle.
- **Data Storage** - There are two variants of data storage - it can either be represented as a rectangle with absence of both smaller sides or as an open-sided rectangle with only one side missing.
- **Data Flow** - Movement of data is shown by pointed arrows. Data movement is shown from the base of arrow as its source towards head of the arrow as destination.

Types of DFD

1. **Context Diagram**:-Level 0 DFDs are also known as context level DFDs. A diagram consisting of only one process which represents the entire system is called context diagram.

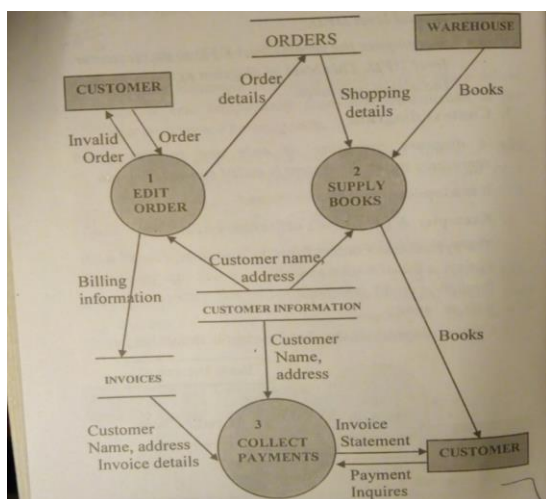
Example:- A publisher's ordering system.

The system takes orders from customers, checks them against a list of books available, verifies customer credit through a credit information file and authorizes supply with an invoice.



2. Detailed – Level DFD:-

A detailed DFD is that where the context level DFD is decomposed to elementary level using DFD symbols.



Advantages of DFD:-

- A straightforward graphical technique which is easy to recognize.
- DFDs can provide a detailed representation of system components.
- It is used as the part of system documentation file.
- DFDs are easier to understand by technical and non technical audiences.

Disadvantages of DFD:-

- It make the programmers little confusing concerning the system.
- The biggest drawback of the DFD is that it simply takes a long time to create.

Data Dictionary:

- A data dictionary is a structured repository of data elements in system. It stores descriptions of all DFD data elements that are **details and definitions of data flows, data stores, and processes.**
- A data dictionary improves the communication between the analyst and the user. It plays an important role in building a database.
- It is an integral part of system specification. Without a data dictionary, DFDs are just pictures without any details.
- Data Dictionary is developed during data flow analysis. It assists the analysts in determining the system requirements.

Need for a Data Dictionary:-

- To manage the details in large systems.
- To communicate a common meaning for all elements.
- To document the features of the system.
- To identify, clarify, define system requirements.
- To evaluate characteristics of the system.

- To build database and write programs.
- To record the details of each element.

Components of a Data Dictionary

1.Data Element:- Data elements are the data at the most fundamental level. These elements are used as building blocks for all other data in the system. Data elements are also referred as data item, elementary item. Data element entry consists of the following items:

- **Data Name**

Unique names are given to data items. Each element is referred by its name in the entire system.

- **Data Description**

It is brief description of what data element represents.

- **Aliases**

Aliases are synonyms or other names for the element. For example Invoice may be called Bill or Price.

- **Length**

Length identifies the number of spaces that are required for each data item.

- **Data Values**

In some processes only specific values are permissible. Example:

Data Name :	<u>Part Colour</u>
Data Description :	<u>This is the colour of the part of the machinery.</u>
Aliases :	<u>P Colur, Colour</u>
Length :	<u>Text of Length 25</u>
Data Value :	<u>May take the following values Red, Yellow, Blue, Orange, Pink</u>

2.Data Structure:-A group of data elements forms a data structure. Data structures are used to define or describe the system components.

Example:- Data Structure of employee consist of data elements like employee code, name, age, experience.

Advantages of Data Dictionary

- It is used to locate errors in the system descriptions,
- It helps user and system analyst to understand all inputs, outputs and components of system.
- It is used to build database.

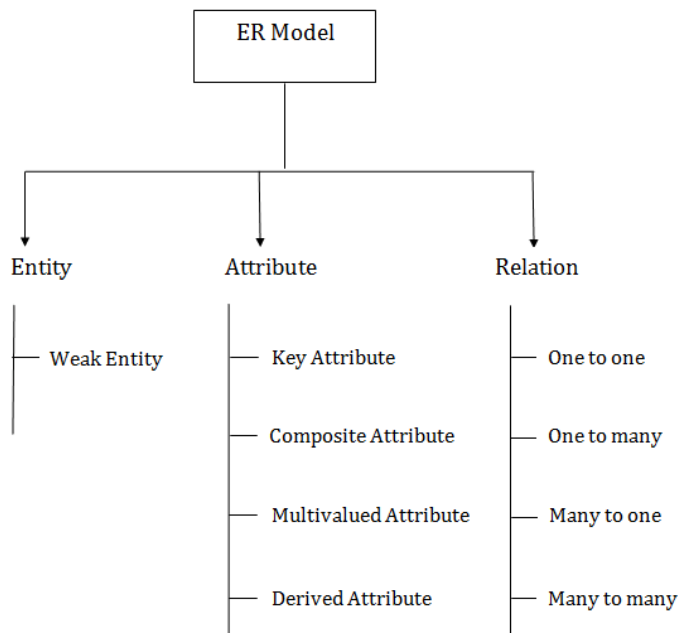
Disadvantages of Data Dictionary

- Data dictionary is tedious and time consuming job.
- It is not acceptable to many non-technical users.

Entity-Relationship Diagrams

- ER model stands for an Entity-Relationship model. This model is used to define the data elements and relationship for specified system.
- It develops simple and easy to design view of data.
- In ER modeling database structure is portrayed as diagram called entity-relationship diagram.
- Used to represent real world scenarios.

Component of ER Diagram



1. Entity:

An entity may be object, class, person or place. In ER diagram, entity is represented as rectangles.

a. Weak Entity

An entity that depends on another entity called a weak entity. It doesn't contain any key attribute. It is represented by a double rectangle.

b. Strong Entity Type:-

Entity types that have key attribute or primary key are called strong entity type.

2. Attribute

The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute. For example, id, age, contact number, name, etc.

a. Key Attribute

It represents a primary key. The key attribute is represented by an ellipse with text underlined.

b. Composite Attribute

Attribute that composed of many other attributes is known as a composite attribute.

c. Multivalued Attribute

Attribute have more than one value are known as a multivalued attribute. The double oval is used to represent multivalued attribute. For example, a student can have more than one phone number.

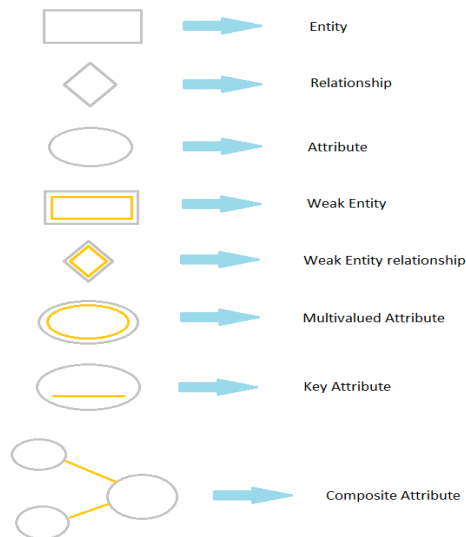
d. Derived Attribute

Attribute that can be derived from other attribute is called derived attribute. Represented by dashed ellipse.

e. Single valued attributes

Attribute that has only single value for an entity is called single valued attribute. For example age.

ER Diagrams symbols and Notations:-



Relationship- A relationship is used to describe relation between entities. Diamond is used to represent relationship.

Types of relationship

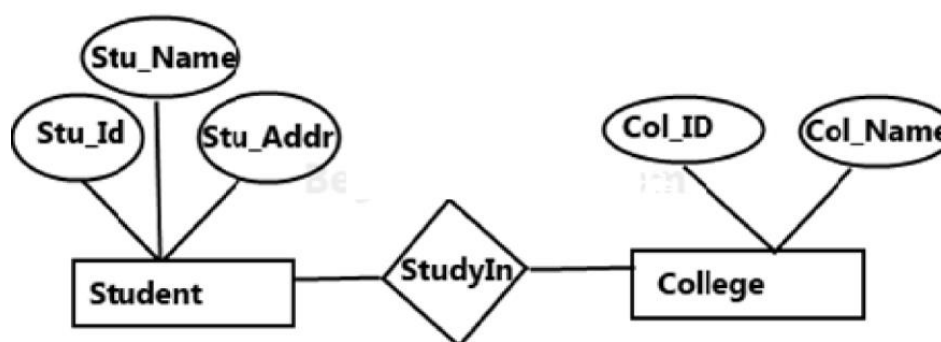
a. One-to-One Relationship-When only one instance of an entity is associated with relationship, it is called one to one relationship.

b. One-to-many relationship-When only one instance of entity on left, and more than one instance of an entity on right associates with relationship is called one-to-many relationship.

c. Many-to-one relationship-When more than one instance of entity on left, and only one instance of an entity on right associates with relationship it is called many-to-one relationship.

d. Many-to-many relationship-When more than one instance of the entity on left, and more than one instance of an entity on right associates with relationship is called many-to-many relationship.

Example of ER Diagram



Software Requirement Specification (SRS):

1. It is a set of documents that contains the concise and clear specification of the functional, performance, design and interface requirements of the proposed system.
2. It is also a set of precisely stated properties or constraints which a software system must satisfy.
4. It is the medium through which the client and user's requirements are accurately specified.

Need of SRS:

1. The basic purpose of SRS is to bridge the communication gap between user and system analyst.
2. SRS establishes the basis of agreement between the client and the software developer.
3. It provides reference for validation of final product.
4. With passage of time, requirements of client keep on changing, so all the requirements are identified with the help of SRS.
5. A high quality SRS leads to high quality software.
6. A high quality SRS reduces problem of rework.

Characteristics of SRS:

1. Completeness:

- SRS should be complete in every way.
- SRS should fully describe functionality to be delivered.
- SRS should contain all types of input.
- SRS should provide features for handling all functions of the system.

2. Correct:

- SRS should be correct.
- SRS must accurately describe functionality to be built.
- Every requirement must be a true requirement of the system.

3. Unambiguous:

- SRS is unambiguous if every stated requirement has only one interpretation.
- SRS should not be confusing.

4. Consistent:

- SRS should be consistent.
- SRS is consistent if and only if no subset of individual requirements described in it is conflict.

5. Modifiable:

- SRS should be modifiable.
- SRS should be changeable as the requirements change while maintaining its completeness and consistency.

6. Traceable:

- SRS is traceable if the origin of each requirement in the SRS must be clear.

7. Verifiable:

- A Requirement is verifiable if and only if there exists some cost-effective process that can check whether the final product meets the requirements.

8. Valid:

- SRS should be valid.
- A valid SRS is that where all parties related to the project can understand and accept it.

Organization/Components of the SRS:

1. Introduction: The Introduction section of the SRS should:

- Describe the purpose of the document.

- Describe the scope of the requirement definition effort.
- Detail the constraints placed upon the requirement process. These includes costs, schedules etc.
- Provide an overview of the entire product.

2. General Description: The General Description section of the SRS should:

- Describe the general factors affecting the product and their requirements.
- Describe the relationship of the product with any other product.
- Describe the relationship of the product to its environment.
- Describe the general function of the product.
- Provide the general characteristics of the users.
- Provide the objectives and requirements for the system from the user's point of view.

3. Functional Requirements: The Functional Requirements of the SRS should:

- Clearly describe every function that the system needs to perform along with the corresponding input and output data set.
- Specify the given inputs, validation conditions on inputs, operations to be performed on the input and corresponding output of operations.
- Specify system behaviour for invalid input data.

- The functional requirements of a system should be complete and consistent.

4. External Interface Requirements: The External Interface Requirements of the SRS should:

- Describe all possible interfaces of the software with the users, hardware and with other software for input or output. These interfaces are shared memory, library routines etc.
- Clearly specify user manual with all user commands, screen formats, feedback and user messages.

5. Performance Requirements:

- The requirement related to the system performance must be specified because the quality of software depends on the performance.
- If performance is good and according to the user's expectation, then it fits to the quality standards.

6. Design Constraints: Design Constraints should:

- Specify types of machine, operating system and language supported.
- Specify recovery requirements like what the system should do if some failure occurs.
- Include security requirements like control to access data, use of passwords etc.
- Specify standard report formats, data naming, accounting procedures and audit tracing.